

Lenguaje Python

Cursada 2026

Funciones: argumentos

Seguimos hablando de funciones

```
def my_function(param1, param2):  
    sentences  
    return <expression>
```

- ¿Necesitamos declarar los parámetros de la función?
- La sentencia **return es opcional**. ¿Qué pasa si no la incluimos?

Uno de los ejemplos de la clase pasada:

```
In [ ]: movies = [50, 48, 85, 93, 100]  
  
def average_calculation(movies_duration):  
    """ This function calculates the average of the lengths of the movies  
    movies_duration: is a list with the duration in minutes of the movies  
    """  
    total_movies = len(movies_duration)  
    average_movies = 0 if total_movies == 0 else sum(movies_duration) / t  
    return average_movies
```

```
In [ ]: average_calculation(movies)
```

Tenían una tarea: ¿de qué forma se pasan los parámetros en Python?

Parámetros en Python

Veamos un ejemplo más sencillo: ¿qué podemos observar al ejecutar el código?

```
In [ ]: def my_demo_function1(param):  
    """This function modifies the received parameter."""  
  
    print(f"El valor de param AL INGRESAR a la función es {param}")  
    param = 0  
    print(f"El valor de param DENTRO de la función es {param}")
```

```
In [ ]: number = 10  
my_demo_function1(number)  
print(f"LUEGO de invocar a la función el valor de num es {number}")
```

Y ahora analicemos este otro ejemplo:

```
In [ ]: def my_demo_function2(param):  
        """This function updates the first position of the list received as a  
        print(f"El valor de param AL INGRESAR a la función es {param}")  
        param[0] = "cero"  
        print(f"El valor de param DENTRO de la función es {param}")  
  
In [ ]: my_list = [100, 200, 300]  
        my_demo_function2(my_list)  
        print(f"LUEGO de invocar a la función el valor de lista es {my_list}")
```

Entonces, ¿qué podemos decir sobre el pasaje de parámetros en Python?

Quando pasamos un parámetro a una función, pasamos **una copia de la referencia** al objeto pasado

Miremos este otro ejemplo

```
In [ ]: def my_demo_function3(param):  
        """This function updates the first position of the list received as a  
        new_list = param[:]  
        print(f"El valor de param AL INGRESAR a la función parametros_colecci  
        new_list[0] = "cero"  
        print(f"El valor de param DENTRO de la función parametros_colecciones  
  
In [ ]: my_list = [100, 200, 300]  
        my_demo_function3(my_list)  
        print(f"LUEGO de invocar a la función el valor de lista es {my_list}")
```

¿Qué pasa? ¿Trabajamos con una copia?

¡Atención! Recordemos qué pasa estructuras que contienen elementos indexados.

DESAFÍO 1

Queremos definir una función que, dada una cadena de caracteres, retorne la **cantidad de vocales abiertas, vocales cerradas y la cantidad total de caracteres** de la misma.

¿Podemos retornar más de un valor?

Una posible solución

¿Qué tipo de dato retorna la función?

```
In [ ]: def process_sentence(sentence):
        """ This function ... """

        sentence = sentence.lower()
        aeo = sentence.count("a") + sentence.count("e") + sentence.count("o")
        iu = sentence.count("i") + sentence.count("u")
        return aeo, iu, len(sentence)
```

```
In [ ]: my_sentence = process_sentence("Python 2026")
        type(my_sentence)
```

¿Cómo accedemos a los valores retornados?

- En el **return** se devuelve una tupla, por lo tanto, accedemos como en cualquier tupla:

```
In [ ]: open_vowels = process_sentence("Python 2026")[0]
        open_vowels
```

```
In [ ]: open_vowels, closed_vowels, len_sentence = process_sentence("1976-2026 50
        open_vowels
```

Veamos algo muy interesante

Observemos esta estructura: ¿de qué tipo es?

```
In [ ]: music = {"bart": {"internacional": ["AC/DC", "Led Zeppelin", "Bruce Sprin
                        "nacional": ["Pappo", "Miguel Mateos", "Los Piojos", "N
                        },
                    "lisa": {"internacional": ["Ricky Martin", "Maluma"],
                        "nacional": ["Lali", "Tini", "Wos"]
                    }
        }
```

Ahora observemos esta función:

En particular, analicemos la lista de argumentos. ¿Notan algo?

```
In [ ]: def my_music(music, name, music_type="nacional"):
        """This function ... """
        if name in music:
            user = music[name]
```

```

        for elem in user[music_type]:
            print(elem)
    else:
        print(f"¡Hola {name}! No tenés registrada música en esta colección")

```

```
In [ ]: my_music(music, "lisa", "internacional")
```

Python permite definir parámetros con valores por defecto

Si hay más de un argumento, los que tienen **valores por defecto siempre van al final de la lista de parámetros**.

```
In [ ]: def demo_function(param1, param2 = "Hola"):
        print(f"{param1 = } - {param2 = }")
```

```
In [ ]: demo_function(10)
```

Los parámetros formales y reales se asocian de acuerdo al **orden posicional**, pero es posible invocar a la función con los parámetros en **otro orden** pero **nombrando al parámetro**.

```
def my_music(music, name, music_type="nacional"):
    """This function ... """

```

```
In [ ]: my_music(music, music_type="internacional", name="lisa")
```

¿Recuerdan este ejemplo? Observemos el print

```
In [ ]: word = "casa"

        for letter in word:
            print(letter, end=" ")

```

```
In [ ]: help(print)
```

Un último ejemplo

```
In [ ]: processing_type = "invertido"
```

```
In [ ]: def show_string(sentence, order=processing_type):
        """ This function returns the string according to the order parameter

        return sentence[::-1] if order == "invertido" else sentence[:]

```

```
In [ ]: show_string("Hola")
```

```
In [ ]: processing_type = "normal"
```

¿Qué pasa ahora si vuelvo a invocar a la función?

```
In [ ]: show_string("Hola")
```

Los valores por defecto de los parámetros se evalúan **una única vez** cuando se define la función.

DESAFÍO 2

Queremos escribir una función que imprima los argumentos que recibe agregando de qué tipo son.

¿Qué tiene de distinta esta función respecto a las que vimos antes o conocemos de otros lenguajes? Por ejemplo:

```
my_args(1)          --> 1 es de tipo <class 'int'>
my_args(2, "hola")  --> 2 es de tipo <class 'int'>,
                        hola es de tipo <class 'str'>
my_args([1,2], "hola", 3.2)
                        --> [1, 2] es de tipo <class 'list',
                        hola es de tipo <class 'str'>,
                        3.2 es de tipo <class 'float'>
```

¿Opciones?

```
In [ ]: def my_args(par1):
        print(type(par1))
        def my_args(par1, par2):
            print(type(par1), type(par2))
        def my_args(par1, par2, par3):
            print(type(par1), type(par2), type(par3))

my_args(1,2,3)
```

Esto no funciona.

Podemos definir funciones con un número variable de parámetros

```
In [ ]: def my_args(*args):
        """ This function..."""

        for value in args:
            print(f"{value = } es de tipo {type(value)}")
```

- **args** es una **tupla** que representa a los parámetros pasados.

```
In [ ]: my_args(1)
print("-"*30)
my_args(2, "hola")
print("-"*30)
my_args([1,2], "hola", 3.2)
```

Otra forma de definir una función con un número variable de parámetros

```
In [ ]: def my_args1(**kwargs):
        """ This function ..... """

        for key, value in kwargs.items():
            print(f"{key = } tiene {value = }")
```

- **kwargs** es un **diccionario** que representa a los parámetros pasados.

```
In [ ]: my_args1(band1= 'Nirvana', band2="Foo Fighters", band3="AC/DC")
```

También podemos tener lo siguiente:

```
In [ ]: def show_data(par1, par2, par3):
        print(par2)

sequence = (1, 2, 3)
show_data(*sequence)
```

- ¿De qué tipo es **sequence**?
- **CONSIGNA:** probar en casa si es posible utilizar otras secuencias vistas.

Otra forma

```
In [ ]: def show_contact(name, phone):
        #print(type(phone))
        print(name, phone)

contact = {"name": "Messi", "phone": 12345}
show_contact(**contact)
```

- ¿De qué tipo es **contact**?
- Observar el nombre de los parámetros: ¿qué podríamos decir?

CONSIGNA: probar estos ejemplos:

```
In [ ]: def show_elements1(param1, param2, param3, param4):
        """Imprimo los valores de los dos primeros parámetros"""
        print( f"{param1}, {param2}")

        def show_elements2(*arguments):
            """Imprimo los valores de los argumentos"""
            for value in arguments:
                print( value)

        def show_elements3(**arguments):
            """Imprimo una tabla nombre-valor"""
            for name, value in arguments.items():
                print( f"{name} = {value}")
```

```
In [ ]: numbers = { "uno": 1, "dos": 2, "tres":3, "cuatro": 4}

print("Invoco a show_elements3 con numbers como parámetro")
show_elements3(**numbers)
print("-" * 20)

print("Invoco a show_elements3 con los parámetros nombrados")
show_elements3(param1 =1, param2 = 2, param3 = 3, param4 = 4)
print("-" * 20)

print("Invoco a show_elements1 con parámetros nombrados")
show_elements1(param1 ="I", param2 = "II", param3 = "III", param4 = "IV")
```

```
In [ ]: print("Invoco a show_element1 con parámetros simples")
show_elements1("I", "II", "III", "IV")

print("-" * 20)

print("Invoco a show_elements2 con parámetros simples")
show_elements2(1, 2, 3, 4)
```

CONSIGNA: ¿todo junto se puede?

Probar este ejemplo y analizar el orden en el que definimos los parámetros.

```
In [ ]: def show_more_values(initial_message, *in_other_language, **in_detail):
        print("Mensaje original")
        print(initial_message)

        print("\nEn otros idiomas")
        print("-" * 40)
        for value in in_other_language:
```

```

    print(value)
    print("\nEn detalle")
    print("-" * 40)

    for key in in_detail:
        print(f"{key}: {in_detail[key]}")
    print("\nFuente: traductor de Google. ")

```

```

In [ ]: show_more_values("Hola",
    "hello", "Hallo", "Aloha ", "Witam", "Kia ora",
    ingles= "hello",
    aleman="Hallo",
    hawaiano="Aloha",
    polaco="Witam",
    maori="Kia ora")

```

Organizar los parámetros

Parámetros posicionales fijos en Python 3.8+

```

In [ ]: def dividir(a, b, /, *, exacto=False):
    # a y b solo posicionales; exacto solo por nombre
    return a / b if exacto else a//b

dividir(10, 3, exacto=True)
#dividir(a=10, b=2)           # error: a y b solo posicionales

```

Controlar con precisión cómo se pueden pasar los argumentos a una función

- Se utilizan los símbolos `/` y `*` en la definición de parámetros.
- **Parámetros posicionales exclusivo** (antes de `/`): los argumentos para estos parámetros deben pasarse únicamente por posición. No se puede usar el nombre del parámetro en la llamada.
- **Parámetros con nombre exclusivos** (después de `*`): los argumentos para estos parámetros deben pasarse obligatoriamente usando su nombre (keyword arguments). No se pueden pasar de forma posicional.

Variables locales y globales

```

In [ ]: z = 5
x = 6
a = 7

def my_function(a):
    global x
    z = 8
    x = 9
    a = 10

```

```
my_function(a)
print(z)
print(x)
print(a)
```

- Variables locales enmascaran las globales.
- Acceso a las globales mediante **global**.

No es una buena práctica utilizar variables globales

Espacio de nombres

Un espacio de nombres **relaciona nombres con los objetos que referencian**.

- Cuando se invoca a una función, se crea un espacio de nombres **local** con todos los recursos definidos en la función y luego se elimina cuando la función finaliza su ejecución.

Volveremos a este tema más adelante...

ATENCIÓN: ¿qué pasa en los siguientes ejemplos?

```
In [ ]: x = 12

def my_function1():
    temp = x + 1
    print(temp)

def my_function2():
    x = x + 1
    print(x)
```

```
In [ ]: my_function1()
```

¿Cuál es el problema al invocar a my_function2?

- Si una variable solo se **lee dentro** de una función y no se le asigna nada, Python la toma del ámbito exterior.
- Si se **asigna** un valor a una variable dentro de una función (con =), esa variable se considera local, y cualquier lectura de su valor antes de la asignación produce error.

Python permite definir funciones anidadas

```
In [ ]: def one():
        def one_one():
            print("uno_uno")
        def one_two():
            print("uno_dos")

        print("uno")
        one_one()

    def two():
        print("dos")
        one_two()

    one()
```

¿Cuál es el problema al invocar a la función two?

¿Qué imprimimos en este caso?

```
In [ ]: x = 0
def one():
    def one_one():
        nonlocal x
        #global x
        x = 100
        print(f"En one_one: {x=}")

    x = 10
    one_one()
    print(f"En one: {x=}")

one()
print(f"En ppal: {x=}")
```

- **global** y **nonlocal** permiten acceder a variables no locales a una función.

Recordemos el Zen de Python ...

```
...
Simple es mejor que complejo.
...
Plano es mejor que anidado.
...
Espaciado es mejor que denso.

La legibilidad es importante.
...
```

¿Entonces?

Observemos este ejemplo

```
In [ ]: def send_message(message, to_person, from_person="alguien", subject="Cons
        """ This function displays a message...
        Arguments:
        to_person: string that represents the recipient of the message
        from_person: string that represents the person who sends the message
        subject: that represents the subject of the message
        """

        print(f"""
            Origen: {from_person}
            Destino: {to_person}
            Asunto: {subject}
            Mensaje: {message}
            """)

send_message("¿Cuáles son las formas de pasar parámetros a una función en
```

Atributos de las funciones

Las funciones en Python también son **objetos**.

- Algunos de sus atributos:
 - **my_function.__doc__**: es el **docstring**.
 - **my_function.__name__**: es una cadena con el nombre la función.
 - **my_function.__defaults__**: es una tupla con los valores por defecto de los parámetros opcionales.

```
In [ ]: print(send_message.__doc__)
        print(send_message.__defaults__)
        print(send_message.__name__)
```

Un artículo sobre estilo de código

<https://realpython.com/python-pep8/>

